SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Concept Mapping

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	**Assessment:**	Assessment Tool 2 – Concept Mapping
Weightage:	35% of CIA	**Total Marks:**	100
CO1 Statement:	*Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.*		
Student Name:	R DINESH	**Reg. No.:**	192524130
Section / Batch:	CSE	**Date:**	08/07/2026

Code of Conduct

I R DINESH(192524130)______________________________(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ______________________

Instructions

- This test contains 5 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should be with following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Linked data structure	Basic data structure	1	20
Primitive and Non-Primitive Data Structure - Linear and Non-Linear Data Structure	Classifications of Linear data structure	2	20
Search Data Structure	Linear Search and Binary Search	3	20
Recursion, Performance analysis	Recursion	4	20
Tree Data Structures	Classifications of Tree data structure	5	20
	GRAND TOTAL:	**100 Marks**	

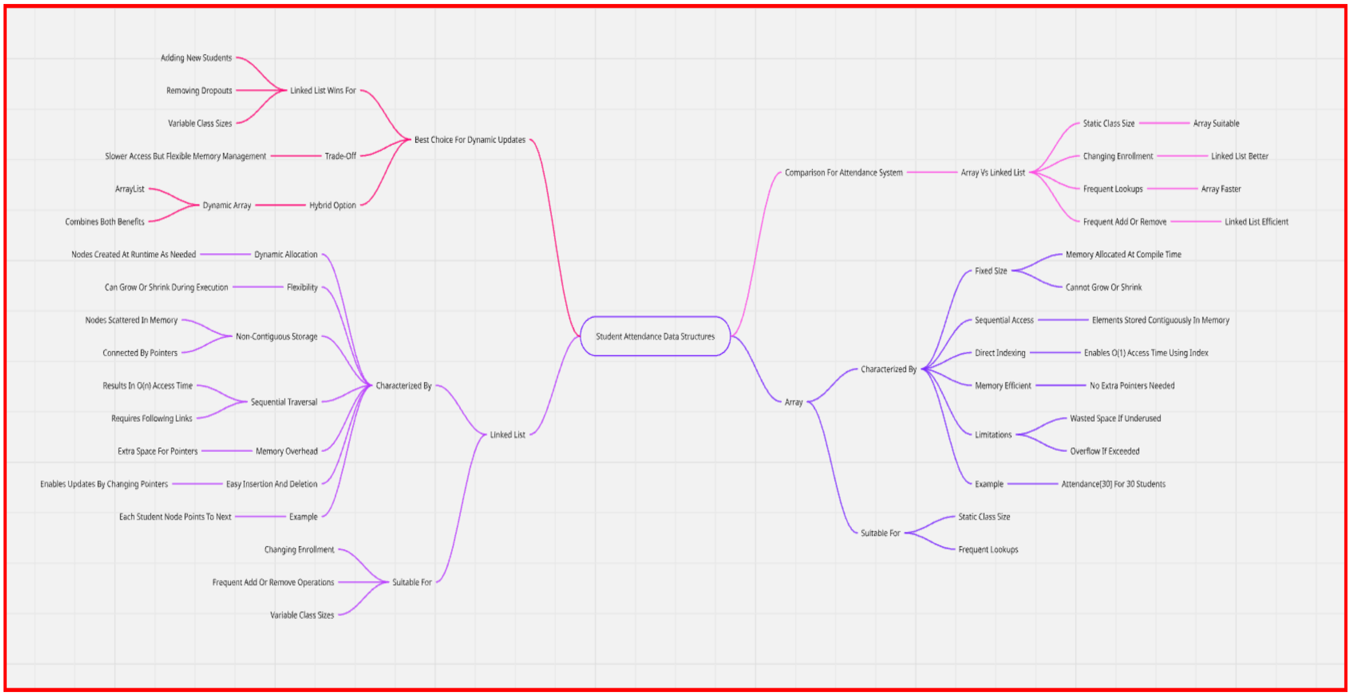
Annexure A – Concept Mapping

1. A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency. (20 Marks)
2. A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance. (20 Marks)
3. E-commerce platform manages product storage, searching, and recommendation systems. Construct a concept map linking Linear Data Structures → Sequential Access → Limitations → Non-Linear Data Structures → Hierarchical/Network Relationships → Applications. Analyze the differences between linear and non-linear structures and demonstrate how each supports specific components of the system. Justify which structure is more efficient for handling complex applications. (20 Marks)
4. A video streaming application must balance speed and memory usage. Construct a concept map connecting Time Complexity → Space Complexity → Tradeoff → Algorithm Design → Performance Optimization. Analyze how improving one factor affects the other and justify the optimal balance. (20 Marks)
5. A digital archive stores millions of documents. Construct a concept map linking Searching → Linear Search → Binary Search → Sorted Data → Time Complexity → Performance. Analyze the impact of data organization on search efficiency and justify the best technique.(20 Marks)

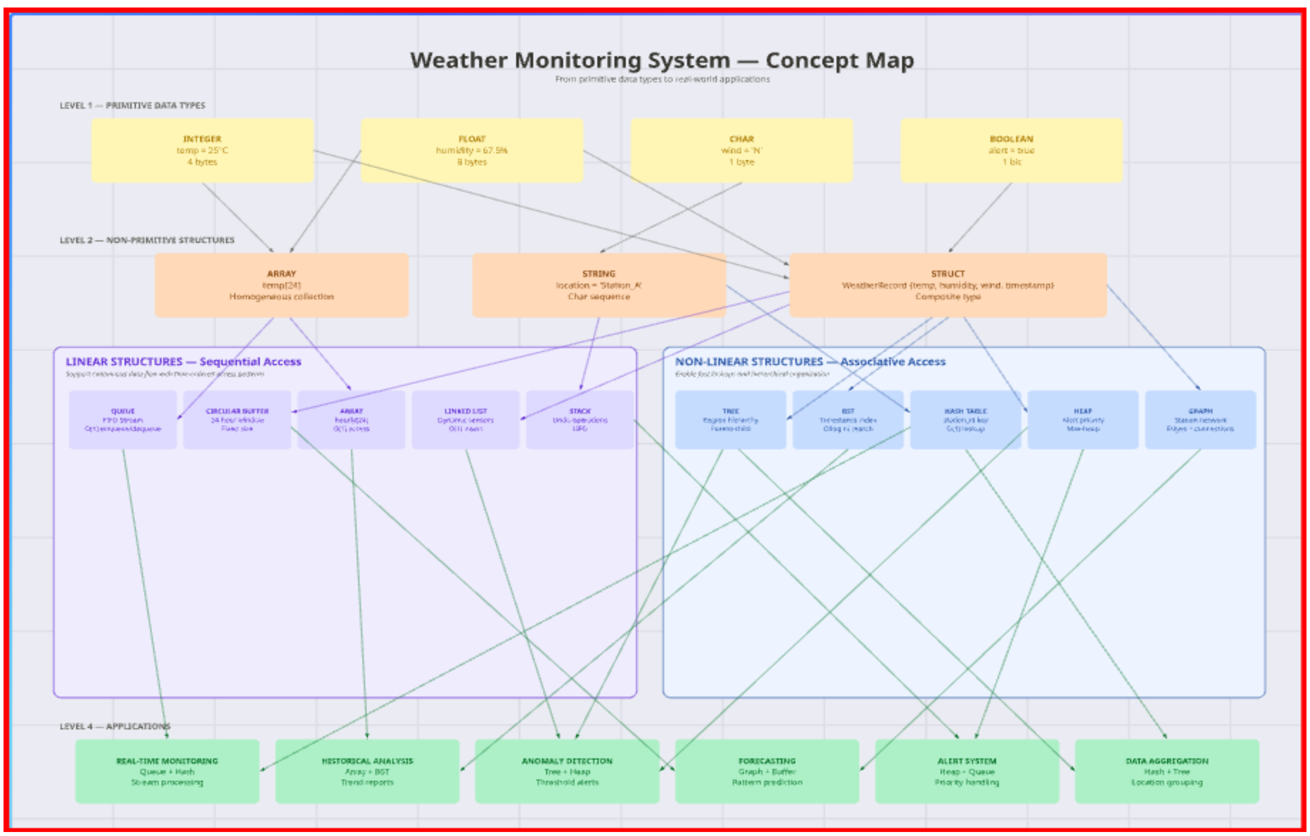
Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: _______________	Signature: _______________	Signature: _______________
Date: ___________________	Date: ___________________	Date: ___________________

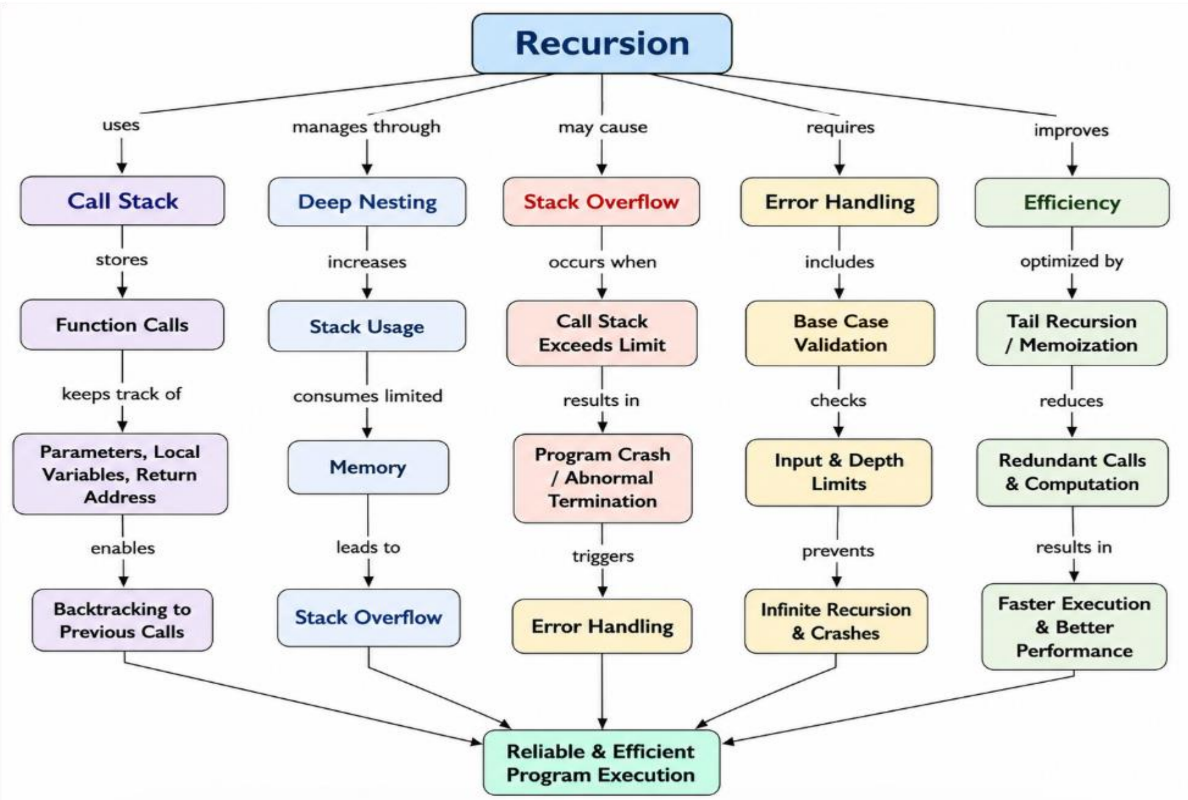


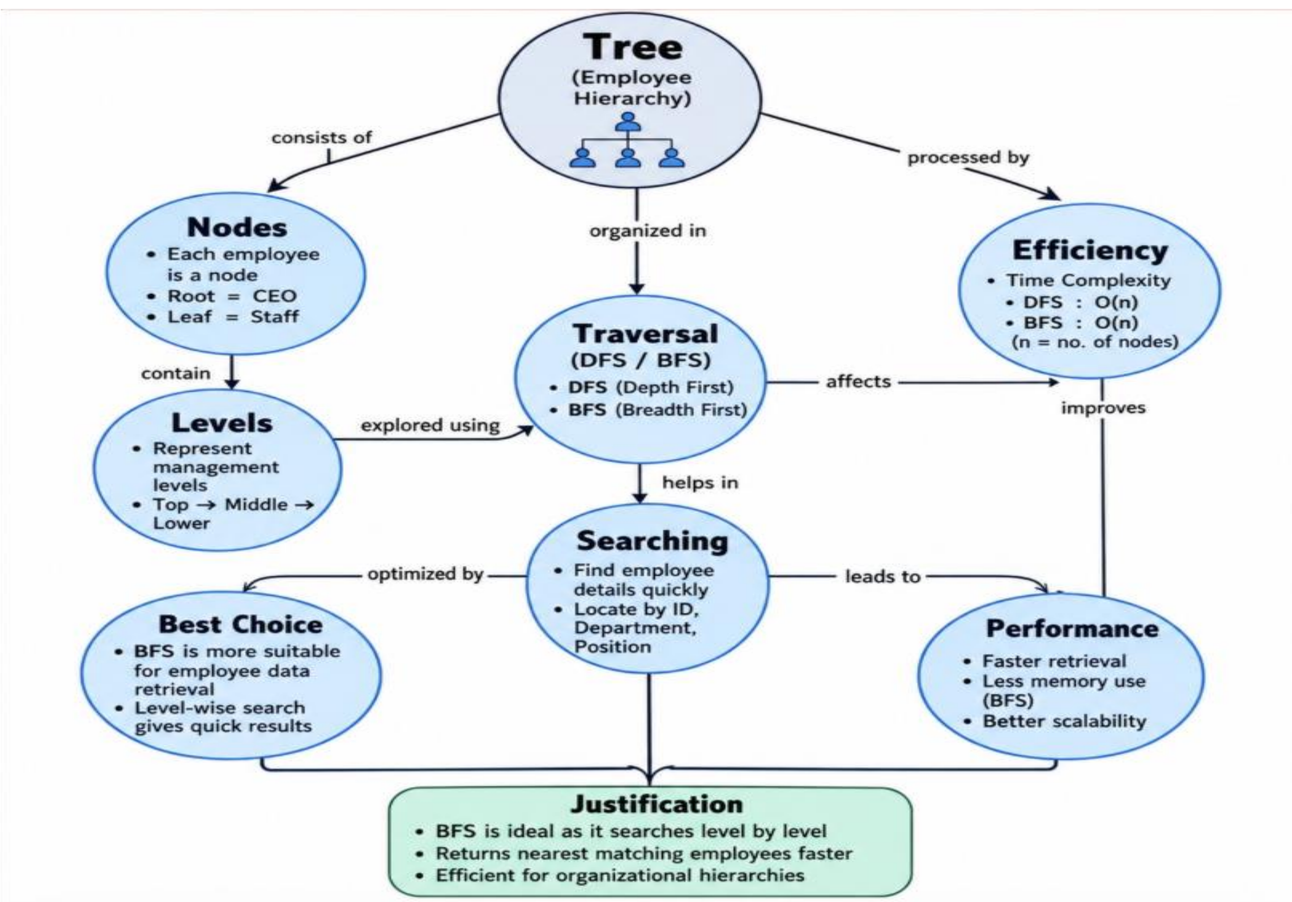
Student Attendance Data Structures
Best Choice For Dynamic Updates
Linked List Wins For
Adding New Students
Removing Dropouts
Variable Class Sizes
Trade-Off
Slower Access But Flexible Memory Management
Hybrid Option
Dynamic Array
ArrayList
Combines Both Benefits
Linked List
Characterized By
Dynamic Allocation
Nodes Created At Runtime As Needed
Flexibility
Can Grow Or Shrink During Execution
Non-Contiguous Storage
Nodes Scattered In Memory
Connected By Pointers
Sequential Traversal
Results In O(n) Access Time
Requires Following Links
Memory Overhead
Extra Space For Pointers
Easy Insertion And Deletion
Enables Updates By Changing Pointers
Example
Each Student Node Points To Next
Suitable For
Changing Enrollment
Frequent Add Or Remove Operations
Variable Class Sizes
Comparison For Attendance System
Array Vs Linked List
Static Class Size
Array Suitable
Changing Enrollment
Linked List Better
Frequent Lookups
Array Faster
Frequent Add Or Remove
Linked List Efficient
Array
Characterized By
Fixed Size
Memory Allocated At Compile Time
Cannot Grow Or Shrink
Sequential Access
Elements Stored Contiguously In Memory
Direct Indexing
Enables O(1) Access Time Using Index
Memory Efficient
No Extra Pointers Needed
Limitations
Wasted Space If Underused
Overflow If Exceeded
Example
Attendance[30] For 30 Students
Suitable For
Static Class Size
Frequent Lookups



Weather Monitoring System — Concept Map
From primitive data types to real-world applications
LEVEL 1 — PRIMITIVE DATA TYPES
INTEGER
temp = 25°C
4 bytes
FLOAT
humidity = 67.5%
8 bytes
CHAR
wind = 'N'
1 byte
BOOLEAN
alert = true
1 bit
LEVEL 2 — NON-PRIMITIVE STRUCTURES
ARRAY
temp[24]
Homogeneous collection
STRING
location = 'Station_A'
Char sequence
STRUCT
WeatherRecord {temp, humidity, wind, timestamp}
Composite type
LINEAR STRUCTURES — Sequential Access
QUEUE
CIRCULAR BUFFER
ARRAY
LINKED LIST
STACK
NON-LINEAR STRUCTURES — Associative Access
TREE
BST
HASH TABLE
HEAP
GRAPH
LEVEL 4 — APPLICATIONS
REAL-TIME MONITORING
Queue + Hash
Stream processing
HISTORICAL ANALYSIS
Array + BST
Trend reports
ANOMALY DETECTION
Tree + Heap
Threshold alerts
FORECASTING
Graph + Buffer
Pattern prediction
ALERT SYSTEM
Heap + Queue
Priority handling
DATA AGGREGATION
Hash + Tree
Location grouping


Searching Techniques in Digital Archive
How data organization affects search efficiency and performance
1 Searching
Find a required document from a large collection.
2 Linear Search
• Checks each document one by one.
• Works on both sorted and unsorted data.
3 Binary Search
• Repeatedly divides the search space into half.
• Faster than linear search.
4 Sorted Data
10 20 30 40 50 60 ...
• Binary search requires documents to be arranged in sorted order.
5 Time Complexity
• Linear Search → O(n)
• Binary Search → O(log n)
log n grows much slower than n as data size increases.
6 Performance
• Binary search gives quicker results for large datasets.
• Linear search becomes slower as data size increases.
7 Data Organization
• Well-organized and sorted data improves search efficiency.
• Unsorted data limits the use of binary search.
8 Best Technique
Binary Search is the best choice for a digital archive with millions of sorted documents because it has O(log n) time complexity and provides high performance.


Recursion
uses
manages through
may cause
requires
improves
Call Stack
Deep Nesting
Stack Overflow
Error Handling
Efficiency
stores
increases
occurs when
includes
optimized by
Function Calls
Stack Usage
Call Stack Exceeds Limit
Base Case Validation
Tail Recursion / Memoization
keeps track of
consumes limited
results in
checks
reduces
Parameters, Local Variables, Return Address
Memory
Program Crash / Abnormal Termination
Input & Depth Limits
Redundant Calls & Computation
enables
leads to
triggers
prevents
results in
Backtracking to Previous Calls
Stack Overflow
Error Handling
Infinite Recursion & Crashes
Faster Execution & Better Performance
Reliable & Efficient Program Execution


Tree
(Employee Hierarchy)
consists of
processed by
organized in
Nodes
• Each employee is a node
• Root = CEO
• Leaf = Staff
Efficiency
• Time Complexity
• DFS : O(n)
• BFS : O(n)
(n = no. of nodes)
contain
Traversal
(DFS / BFS)
• DFS (Depth First)
• BFS (Breadth First)
affects
improves
Levels
• Represent management levels
• Top → Middle → Lower
explored using
helps in
Searching
• Find employee details quickly
• Locate by ID, Department, Position
optimized by
leads to
Best Choice
• BFS is more suitable for employee data retrieval
• Level-wise search gives quick results
Performance
• Faster retrieval
• Less memory use (BFS)
• Better scalability
Justification
• BFS is ideal as it searches level by level
• Returns nearest matching employees faster
• Efficient for organizational hierarchies